William Jackson

Senior Software Engineer – PHP (Laravel/Symfony) AWS/Azure

Wellington, Nevada, United States | +1 (786) 949-7561| williamjackson.pro@outlook.com | linkedin.com/in/william-jackson-75b9343b6

PROFESSIONAL SUMMARY

Senior **Backend Engineer** with **10+ years** of experience building high-availability **PHP 5.6–8.3** services and API platforms across **SaaS, enterprise workflow, operations, and integration-heavy product environments** on **AWS-first** architectures. Strong expertise in **Laravel 5.8–10**, **Symfony 3.4–6**, distributed service design, secure REST and GraphQL delivery, and event-driven processing with **Kafka**, **RabbitMQ**, and **AWS SQS/SNS**. Deep background in **PostgreSQL**, **MySQL**, **SQL Server**, **Redis**, and **Elasticsearch/OpenSearch**, with proven success improving latency, release reliability, and production visibility through **OpenTelemetry**, **Datadog**, **Prometheus/Grafana**, and **CloudWatch** while enforcing **OAuth2/OIDC**, **JWT**, **RBAC/ABAC**, and audit logging standards.

SKILLS

- **Backend & APIs** — **PHP 5.6–8.3**, **Laravel 5.8–10**, **Symfony 3.4–6**, REST APIs, GraphQL, API gateway/BFF patterns, microservices, event-driven architecture, background jobs, WebSockets, idempotency, rate limiting, secure coding, integration design, legacy modernization
- **Data, Messaging & Search** — **PostgreSQL 10–16**, **MySQL 5.7–8.0**, **SQL Server 2016–2022**, **Redis 5–7**, MongoDB, DynamoDB, **AWS SQS/SNS**, **Kafka**, **RabbitMQ**, **Elasticsearch 7–8**, **OpenSearch 1–2**, Redshift, BigQuery
- **Cloud & DevOps** — **AWS**: EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, DynamoDB, ElastiCache, EventBridge, Step Functions, IAM | **Azure**: AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights | **GCP**: GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery | Docker, Kubernetes, Helm, Argo CD, GitOps, Terraform, CloudFormation | GitHub Actions, GitLab CI, Jenkins, Azure DevOps, CircleCI
- **Observability, Quality & Security** — **OpenTelemetry**, **Datadog**, **Prometheus**, **Grafana**, **ELK/EFK**, **CloudWatch**, structured logging, Jest, Mocha, Chai, Cypress, Playwright, Pact-style contract testing, **OAuth2**, **OpenID Connect**, **JWT**, Okta, Auth0, AWS Cognito, **RBAC/ABAC**, rate limiting, audit logging

WORK HISTORY

Senior Software Engineer — February 2021 to December 2025
Viaro Networks Inc. - Delaware, United States

- Led a major backend modernization program that decomposed a tightly coupled legacy monolith into **PHP 7.4–8.2** domain services, which improved engineering ownership boundaries and reduced cross-team dependency delays by **35%** during release coordination.
- Designed and shipped high-volume **REST APIs** with standardized validation, pagination, and error contracts, which cut client-side integration defects by **30%** and made production troubleshooting significantly faster for internal and partner teams.
- Implemented resilient asynchronous workflows using **AWS SQS/SNS** with retry policies, dead-letter queues, and replay-safe job handlers, solving repeated timeout and partial-processing issues that had been causing unstable long-running operations.
- Introduced **Kafka**-based event streaming for high-throughput business events and fixed duplicate-processing defects by enforcing **idempotency keys**, consumer de-duplication logic, and safe replay controls across critical downstream services.
- Optimized **PostgreSQL** performance through index redesign, execution-plan analysis, and query refactoring, improving p95 latency by **40%** on several heavily used API endpoints that had become slow under concurrent traffic.
- Maintained compatibility across inherited **MySQL** and **SQL Server** modules, then standardized schema migration and rollback procedures to eliminate deployment drift and resolve recurring data inconsistency issues after environment promotions.
- Implemented **Redis** caching through **ElastiCache** with disciplined TTL rules, key naming, and invalidation flows, reducing peak database pressure by **28%** and stabilizing high-read workloads during customer traffic spikes.

- Delivered flexible persistence patterns using **MongoDB** for semi-structured operational payloads and **DynamoDB** for latency-sensitive access paths, which solved schema-friction problems without weakening core relational data integrity.
- Built entity indexing pipelines into **Elasticsearch 7–8** and **OpenSearch 1–2**, enabling low-cost filtering, audit-friendly retrieval, and faster internal triage workflows while reducing the reporting burden on transactional databases.
- Orchestrated cross-service workflows with **EventBridge** and **Step Functions**, separating orchestration logic from business services and fixing brittle multi-step failure scenarios that previously required manual operational recovery.
- Hardened platform security with **OAuth2**, **OpenID Connect**, **JWT**, and **RBAC/ABAC**, while adding consistent **audit logging** standards that improved traceability for privileged actions and simplified security review discussions.
- Rolled out **OpenTelemetry** tracing and unified observability with **Datadog**, **Prometheus**, **Grafana**, and **CloudWatch**, which reduced mean time to root cause by **45%** by correlating API, queue, and worker execution paths.

Software Engineer — October 2018 to January 2021
Avantia, Inc. - Ohio, United States

- Developed reliable **PHP 7.2–7.4** backend services for operational workflows, focusing on durable writes, safe retry behavior, and predictable failure handling that reduced production-side processing errors across core internal transactions.
- Implemented stable **REST** endpoints with carefully controlled payload shapes and semantic consistency, which lowered integration regressions and helped partner teams consume APIs without repeated clarification or contract mismatches.
- Built queue-driven background processing with **RabbitMQ** using retry, dead-letter, and poisoned-message handling patterns, solving job-stall incidents that had previously caused silent failures and manual recovery work.
- Expanded event-driven integrations with **Kafka** to distribute domain events across billing, analytics, and notification services, which improved decoupling and reduced direct point-to-point integration maintenance across teams.
- Strengthened persistence across **PostgreSQL**, **MySQL**, and **SQL Server** by performing index audits, remediating slow queries, and tightening migration practices, which improved write stability and reduced lock-contention incidents.
- Added **Redis** caching for high-frequency read paths and standardized TTL, invalidation, and cache-key conventions, preventing stale-data issues that had previously created difficult-to-reproduce bugs in downstream workflows.
- Delivered search-driven investigation tooling with **Elasticsearch**, allowing support and operations teams to triage records quickly without overloading primary transactional queries during business-critical reporting windows.
- Implemented **OAuth2** and **JWT** access controls with consistent role enforcement at the API layer, closing authorization gaps and improving security posture for sensitive workflow actions and partner-facing integrations.
- Instrumented structured logging, metrics, and traces using **OpenTelemetry**, then shipped signals into **Datadog** and **Grafana**, giving the team clearer visibility into latency spikes, queue buildup, and intermittent service errors.
- Centralized operational logs into **ELK/EFK** and connected alerting to documented runbooks, which reduced noisy pages by **25%** and improved on-call response quality during recurring production incidents.
- Supported cloud workloads on **AWS** with **S3**, **RDS**, **DynamoDB**, and **ElastiCache**, while also integrating **Azure Service Bus** and **Key Vault** for specific enterprise needs that required controlled hybrid-cloud interoperability.

Software Developer March 2016 to September 2018
ArnAmy, Inc. - Florida, United States

- Built and maintained **PHP 5.6–7.2** backend modules for internal business platforms, refactoring legacy procedural code into testable service layers that made defects easier to isolate and new feature work safer to deliver.
- Designed transactional workflows with clear write boundaries across **PostgreSQL** and **MySQL**, preventing partial-update failures during reconciliation routines that had previously created manual cleanup work for operations teams.
- Implemented background jobs with retry control, alerting, and safer execution paths, converting fragile manual processes into automated pipelines and reducing repetitive operational effort by **32%** across several internal workflows.
- Introduced **Redis** caching for read-heavy endpoints and dashboard queries, which lowered repeated database access during peak usage and resolved intermittent performance degradation affecting user-facing admin screens.
- Added selective **MongoDB** support for semi-structured records where relational modeling created unnecessary friction, while keeping source-of-truth entities in normalized databases to preserve reporting and operational consistency.
- Integrated external vendor systems through adapter layers and defensive transformation logic, isolating third-party contract volatility and solving repeated upstream format-change issues that had been breaking downstream processes.
- Implemented early search capabilities with **Elasticsearch** indexing for internal entities, enabling much faster admin filtering and support lookups while avoiding expensive wildcard-style queries on transactional data stores.
- Established baseline production monitoring with **CloudWatch** and structured logs, which made recurring incidents reproducible and significantly improved the team's ability to trace failures without depending on tribal knowledge.
- Delivered containerized developer environments with **Docker** and early **Kubernetes** workflows, reducing onboarding inconsistency and fixing the environment drift that had been slowing local setup and release validation.
- Supported CI automation through **Jenkins** pipelines and assisted analytics exports into **Redshift** and **BigQuery**, creating more reliable reporting flows for stakeholders and improving confidence in operational dashboards.

Junior Software Developer October 2014 to February 2016
Centric Tech Inc. - New Jersey, United States

- helping reduce avoidable production surprises and building a strong base for maintainable service behavior.
- Built durable data-access logic on **SQL Server** with transaction-aware write patterns, which improved consistency for business-critical records and reduced the frequency of rollback-related support incidents during busy periods.
- Implemented background job processing using **RabbitMQ** patterns for retry, backoff, and failure isolation, solving reliability issues in long-running tasks that had previously been vulnerable to silent message loss.
- Added basic **Redis** caching for frequently requested endpoints, which lowered unnecessary database pressure and improved responsiveness for common internal workflows used heavily by support and operations teams.
- Implemented internal entity indexing with **Elasticsearch**, enabling faster filtering and record discovery while reducing the load created by broad search queries against transactional tables during high-usage windows.
- Standardized build and deployment routines through **Jenkins** conventions, which reduced environment-specific failures and helped eliminate the recurring "works on my machine" problems that slowed team delivery.
- Supported early **AWS** usage for asset storage and operational services, contributing to **S3-**style persistence flows and basic runbook practices that improved deployment confidence and production readiness.
- Helped introduce structured logging and simple operational dashboards, improving issue diagnosis speed and giving senior engineers clearer context when resolving incidents across shared backend components.

- Collaborated closely with senior engineers through code reviews, pairing sessions, and production debugging, which strengthened engineering discipline and accelerated the ability to solve bugs, delivery blockers, and integration issues.

EDUCATION

Bachelor’s degree in Computer Science 2010 to 2014
Stanford University Department of Computer Science

- Built strong foundations in **algorithms**, **databases**, and **software engineering**, applying coursework through practical team projects and production-style system design.

CERTIFICATIONS

- **AWS Certified Developer – Associate**
  Validated hands-on knowledge of building, deploying, and troubleshooting cloud-native application workloads across core **AWS** services used in modern backend delivery.
- **AWS Certified Solutions Architect – Associate**
  Demonstrated practical understanding of resilient architecture design, service integration patterns, security controls, and cost-aware infrastructure decisions for distributed systems.
- **Certified Kubernetes Application Developer (CKAD)**
  Confirmed working capability in containerized workload design, Kubernetes-native deployment patterns, configuration management, and day-to-day application operations in clustered environments.